

컴퓨터비전(AI응용)

강화학습

강화학습이란?

- 강화학습은 에이전트(Agent)가 환경(Environment)과 상호작용하며 보상(Reward)을 최대화하는 행동 전략을 스스로 학습하는 머신러닝 패러다임입니다.
- 지도학습과 달리 정답 레이블 없이 시행착오를 통해 학습하며, 게임 AI, 로봇 제어, 자율주행, 추천 시스템 등 다양한 분야에서 혁신적인 성과를 만들어내고 있습니다.

핵심 구성 요소

- Agent: 환경에서 행동을 결정하는 학습 주체
- Environment: 에이전트가 상호작용하는 외부 세계
- Reward: 행동에 대한 피드백 신호
- Policy: 상태에서 행동으로의 매핑 전략

실제 적용 사례

- 게임 AI :
 - AlphaGo는 DQN과 몬테카를로 트리 탐색을 결합하여 세계 최고 바둑 기사를 이겼습니다.
 - Atari 게임에서도 인간 수준 이상의 성능을 달성했습니다.
- 로봇 제어 :
 - 산업용 로봇의 조립 작업, 물체 파지, 이동 경로 최적화에 강화학습이 활용되어 효율성과 정확도를 크게 향상시켰습니다.
- 자율 주행 :
 - 복잡한 교통 상황에서의 의사결정, 차선 변경, 장애물 회피 등 실시간 제어 문제를 강화학습으로 해결합니다.
- 추천 시스템 :
 - 사용자의 장기적인 만족도를 최대화하기 위해 콘텐츠, 광고, 제품 추천에 강화학습 기반 개인화 알고리즘 적용

MDP

- Markov Decision Process는 강화학습 문제를 수학적으로 정의하는 프레임워크입니다.
- 현재 상태 만으로 미래를 예측할 수 있다는 Markov 속성을 가정합니다.



State (S)

에이전트가 관찰하는 환경의 현재 상황. 예: CartPole의 각도와 속도



Action (A)

에이전트가 선택할 수 있는 행동의 집합. 예: 왼쪽/오른쪽 이동



Reward (R)

특정 행동의 즉각적인 보상. Policy 학습의 피드백 신호 역할



Transition (P)

행동에 따른 다음 상태로의 전이 확률. $P(s'|s,a)$ 형태로 표현



Discount Factor (γ)

미래 보상의 현재 가치를 결정하는 0~1 사이의 값. 장기적 전략 수립

가치 함수의 개념

- Value Function은 특정 상태 또는 상태-행동 쌍이 얼마나 좋은지를 나타내는 기대 누적 보상입니다.
- $V(s)$: 상태 s 에서 정책 π 를 따를 때의 기대 수익
- $Q(s,a)$: 상태 s 에서 행동 a 를 취한 후 정책 π 를 따를 때의 기대 수익

Bellman 방정식

- 현재 가치와 미래 가치의 재귀적 관계를 정의합니다:

$$V(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')]$$

- 이 방정식은 최적 정책을 찾기 위한 모든 RL 알고리즘의 수학적 기반입니다.

Gym

- Gym은 강화학습 알고리즘을 개발하고 비교하기 위한 표준화된 환경 라이브러리입니다.
- 다양한 난이도의 문제를 제공하여 학습자와 연구자가 쉽게 실험할 수 있습니다.

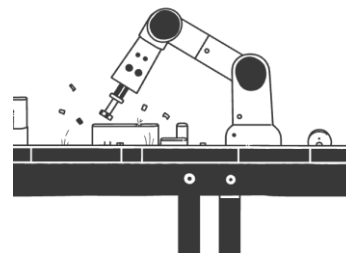


FrozenLake

4×4 또는 8×8 격자에서 시작 지점에서 목표 지점까지 안전하게 이동하는 문제입니다.

얼음 구멍(H)을 피하고 목표(G)에 도달하면 +1 보상을 받습니다.

- State: 16개 또는 64개 위치
- Action: 상하좌우 4방향
- 확률적 전이(미끄러짐)



CartPole

카트 위의 막대를 수직으로 유지하는 고전적인 제어 문제입니다. 막대가 쓰러지지 않고 오래 버틸수록 높은 보상을 받습니다.

- State: 위치, 속도, 각도, 각속도(4차원 연속)
- Action: 왼쪽/오른쪽 힘 적용
- Episode 길이 500 step 목표

강화학습의 핵심 과제는 알려진 좋은 행동을 활용(Exploitation)하면서 동시에 더 나은 행동을 탐색(Exploration)하는 균형을 맞추는 것입니다.



탐색 (Exploration)

새로운 상태와 행동을 시도하여 환경에 대한 지식을 확장합니다.
초기 학습 단계에서 중요합니다.



활용 (Exploitation)

현재까지 학습한 정보를 바탕으로 최선의 행동을 선택하여
보상을 최대화합니다.

ϵ -greedy Policy

가장 널리 사용되는 탐색 전략입니다. ϵ 확률로 무작위 행동을 선택하고, $(1-\epsilon)$ 확률로 현재 최선의 행동을 선택합니다.

- 초기에는 높은 ϵ 값(0.9~1.0)으로 적극 탐색
- 학습이 진행되며 ϵ 를 점진적으로 감소(decay)
- 최종적으로 낮은 ϵ 값(0.01~0.1)으로 수렴하여 주로 활용

Q-learning은 각 상태-행동 쌍의 가치(Q-value)를 테이블에 저장하고 반복적으로 업데이트하여 최적 정책을 학습하는 off-policy 알고리즘입니다.

1. Q-table 초기화

모든 상태-행동 쌍의 Q-value를 0 또는 작은 무작위 값으로 설정

2. 행동 선택

ϵ -greedy 정책으로 탐색과 활용의 균형을 맞추며 행동 선택

3. Q-value 업데이트

Bellman 방정식에 따라 $Q(s,a)$ 값을 점진적으로 개선

4. 수렴

충분한 반복 후 Q-table이 최적 가치 함수에 근사

□ Q-learning 업데이트 규칙

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

여기서 α 는 학습률(learning rate), γ 는 할인 계수(discount factor)입니다.

학습률은 새로운 정보를 얼마나 빠르게 반영할지를 결정하며, 할인 계수는 미래 보상의 중요도를 조절합니다.

DQN

- Deep Q-Network는 Q-learning의 테이블을 신경망으로 대체하여 고차원 상태 공간 문제를 해결합니다.
- Atari 게임에서 인간 수준 성능을 달성하며 딥러닝 기반 강화학습 시대를 열었습니다.



Experience Replay

과거 경험을 메모리에 저장하고 무작위로 샘플링하여 학습. 데이터 간 상관관계를 제거하고 학습 안정성을 높입니다.



Target Network

일정 주기마다 업데이트되는 별도의 네트워크로 목표값 계산.
학습 과정의 발산을 방지합니다.



Deep Neural Network

다층 신경망으로 Q-function 근사.
복잡한 패턴과 고차원 입력(이미지 등)을 효과적으로 처리합니다.

Loss Function

$$L(\theta) = E_{(s,a,r,s') \sim D} [(r + \gamma \max_a Q(s', a; \theta^-) - Q(s, a; \theta))^2]$$

여기서 θ 는 현재 네트워크 파라미터, θ^- 는 Target Network 파라미터, D 는 Replay Buffer입니다.

CartPole DQN 실습 구조



환경 및 하이퍼파라미터 설정

Gymnasium으로 CartPole-v1 환경 생성, 학습률(0.001), 배치 크기(64), 버퍼 크기(10,000) 등 설정



DQN 모델 정의

PyTorch로 4차원 입력(state), 2개 은닉층(128, 128 units), 2차원 출력(Q-values) 구조 구현



Replay Buffer 구현

(state, action, reward, next_state, done) 튜플을 저장하고 랜덤 샘플링하는 메모리 클래스 작성



학습 루프 실행

Episode별 상호작용, ϵ -greedy 행동 선택, Q-network 업데이트, Target network 동기화 반복



결과 시각화

Episode별 reward, 이동 평균, 성공률 그래프를 matplotlib으로 생성하여 학습 과정 분석

Value-based vs Policy-based 비교

강화학습의 두 가지 핵심 접근법의 차이와 특징

Value-based

- ✓ 대표 알고리즘: **Q-Learning, DQN**
- ✓ Q-value 학습 후 **간접적 정책** 도출
- ✓ $Q(s,a) \rightarrow \arg\max \rightarrow$ **결정적(Deterministic)**
- ✓ 주로 **이산 행동 공간**에 적합

Policy-based

- ✓ 대표 알고리즘: **Policy Gradient, PPO**
- ✓ 파라미터화된 정책을 **직접 학습**
- ✓ $\pi(a|s) \rightarrow$ 확률 분포 $\rightarrow$ **확률적(Stochastic)**
- ✓ **연속 행동 공간** 처리에 유리

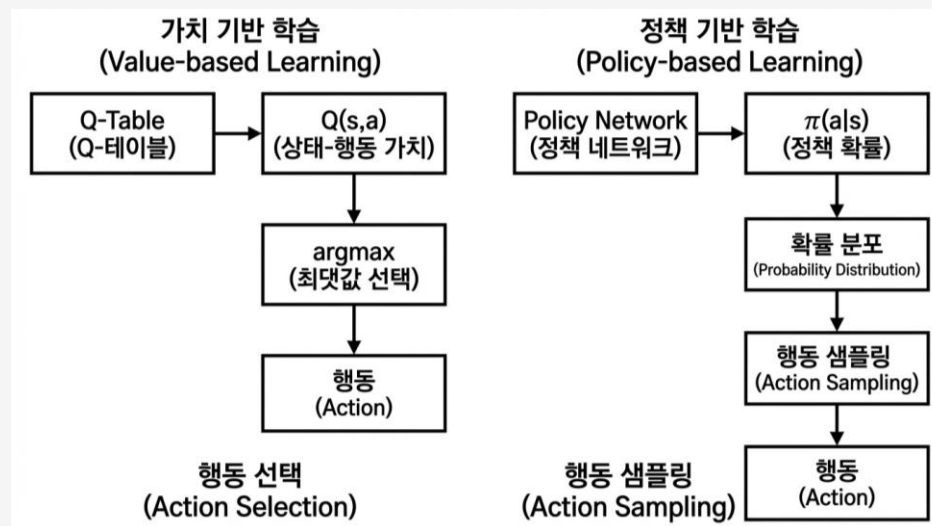


그림 1: 가치 기반 학습(좌)과 정책 기반 학습(우)의 의사결정 흐름 비교

정책 신경망 구조 (Policy Network Architecture)

행동 공간의 특성에 따른 신경망 설계 차이

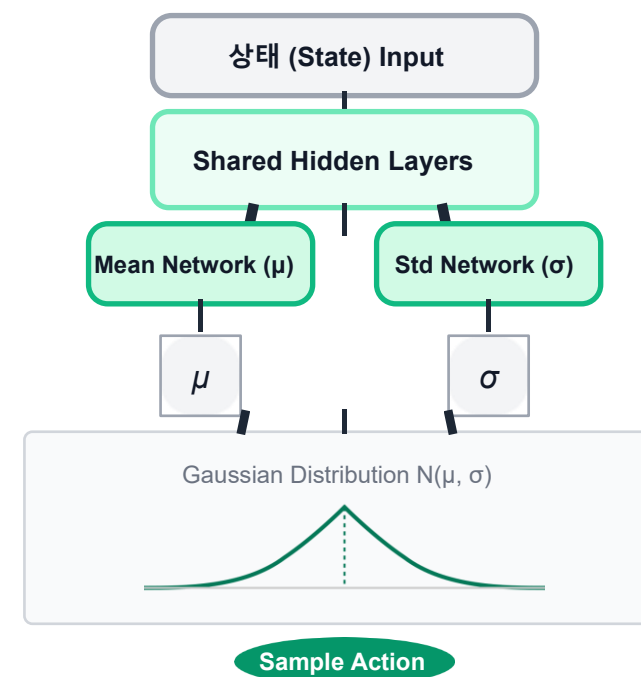
田 이산 행동 (Discrete)

예: CartPole (왼쪽/오른쪽), 아타리 게임 (조이스틱)



ㄱ 연속 행동 (Continuous)

예: 로봇 팔 제어 (관절 각도, 토크), 자율 주행 (조향각)



신경망의 출력층 구조가 행동 공간의 종류(이산/연속)에 따라 달라짐

Policy Gradient Theorem 핵심

강화학습의 목표: 기대 보상을 최대화하는 정책 찾기

목적 함수 총 보상의 기댓값을 최대화하는 것

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

Policy Gradient Theorem

The "Magic" Formula

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right]$$

💡 직관적 해석

↑ 좋은 결과를 낸 행동의 확률을 높인다

↓ 나쁜 결과를 낸 행동의 확률을 낮춘다

"시행착오를 통해 보상이 높은 행동 패턴을 강화하는 과정"

θ

Parameter

정책 신경망의 가중치(Weights)

G_t

Return

시점 t부터의 누적 할인 보상 합

$\nabla \log \pi$

Score Function

행동 확률의 로그 그래디언트

Monte Carlo Policy Gradient: 에피소드 단위 학습

1

에피소드 실행 및 궤적 수집

현재 정책으로 게임 끝까지 플레이

2

할인 리턴(Discounted Return) 계산

각 시점 t 에서의 누적 보상 합계 계산

3

Policy Gradient (Loss) 계산

로그 확률과 리턴의 곱으로 손실 정의

4

파라미터 업데이트

Gradient Ascent (Loss 최소화 방향의 반대)

핵심 특징



Episodic Update

에피소드가 완전히 끝나야만 학습 가능
(Monte Carlo 방식)



High Variance

에피소드마다 보상 편차가 커서 학습이 불안정함



Unbiased

기댓값은 참된 그라디언트에 수렴함 (편향이 없음)

PYTHON PSEUDOCODE

```
optimizer.zero_grad()
loss = -torch.sum(log_probs * returns)
loss.backward()
optimizer.step()
```

REINFORCE 알고리즘 장점과 단점

단순하지만 강력한 기초, 그러나 명확한 한계점 존재



장점 (Pros)

왜 사용하는가?



구현이 매우 간단

복잡한 가치 함수 근사 없이 정책을 직접 최적화



확률적 정책 (Stochastic Policy)

자연스러운 탐색(Exploration)이 가능하며, 가위바위보 같은 문제 해결 가능



연속 행동 공간 처리

DQN과 달리 로봇 제어 등 연속적인 행동을 자연스럽게 다룸



수렴성 보장 (On-policy)

적절한 step size 하에서 로컬 최적점(Local Optima)으로의 수렴이 보장됨



해결책이 필요한 시점!

높은 분산 문제를 해결하기 위해 Baseline과 Critic을 도입해야 합니다.



단점 (Cons)

무엇이 문제인가?



높은 분산 (High Variance)

에피소드마다 보상 차이가 커서 학습이 매우 불안정하고 요동침



샘플 비효율성 (Inefficiency)

데이터를 한 번 쓰고 버려야 함 (On-policy), 과거 경험 재사용 불가



느린 학습 속도

에피소드가 끝날 때까지 기다려야 하며(MC), 많은 데이터가 필요함



Credit Assignment 문제

어떤 특정 행동이 보상에 기여했는지 알기 어려움 (모든 행동에 동일 보상 분배)

왜 분산이 문제인가?

동일한 행동에 대한 불확실한 보상 신호가 학습을 방해함

❧ 동일 행동, 다른 결과

Episode 1

Good Luck

Action: Right

→ Reward: +200

Episode 2

Bad Luck

Action: Right

→ Reward: +50

Episode 3

Normal

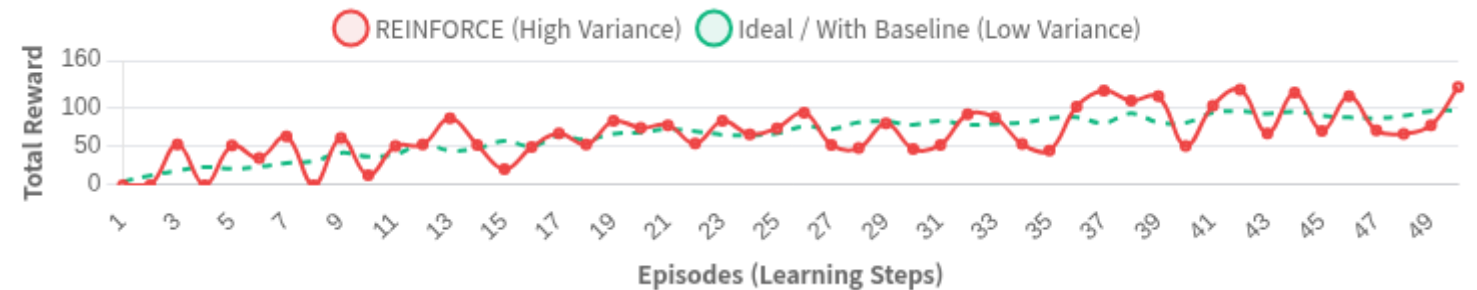
Action: Right

→ Reward: +150

⚠️ 핵심 문제점 Gradient 방향이 계속 바뀜.

같은 상황에서 같은 행동을 했는데 보상이 들쭉날쭉하면, 이로 인해 학습이 매우 불안정하고 수렴이 느려짐.

📈 학습 곡선의 불안정성 (High Variance)



Gradient Noise

파라미터 업데이트 방향이
매 스텝마다 요동침

Slow Convergence

최적 정책을 찾는데
불필요하게 많은 샘플 소모

Local Optima

노이즈로 인해
나쁜 지역 해에 빠질 위험 증가

Baseline 개념 도입

절대적인 보상 크기가 아닌, '평균 대비' 상대적인 성능이 중요하다



상대적 가치 평가

단순히 보상의 절대값(G_t)이 양수라고 해서 무조건 강화하는 것은 비효율적입니다. "평소보다 얼마나 더 잘했는가?"가 핵심입니다.



수식의 변화

$$\nabla J(\theta) \propto (G_t - b) \nabla \log \pi_{\theta}(a_t | s_t)$$



분산 감소 효과

평균보다 좋으면 강화(+), 나쁘면 약화(-)함으로써 학습 신호의 변동성을 줄이고 수렴 속도를 높입니다.

예시: 평균 점수(Baseline) = 125점

Case A: 200점 획득

↑ 확률 증가 (+75)

G=200

"평균보다 잘했으니 이 행동을 더 많이!"

Case B: 50점 획득

↓ 확률 감소 (-75)

G=50

"평균보다 못했으니 이 행동을 줄이자"

최적 Baseline과 불편성 유지

분산을 최소화하면서도 학습 방향(Gradient)은 왜곡하지 않는 방법

최적 Baseline 상태 가치 함수 $V(s)$ 를 Baseline으로 사용할 때 분산이 최소화됨

$$b^*(s_t) \approx V^\pi(s_t)$$

Baseline b 는 상태 s_t 에만 의존해야 함 (행동 a_t 와 무관)

불편성 증명 (Unbiased) Baseline을 빼도 기댓값(Gradient 방향)은 변하지 않음

$$\mathbb{E}_\pi [\nabla_\theta \log \pi_\theta(a_t | s_t) \cdot b(s_t)] = 0$$

$$\therefore \nabla J(\theta) = \mathbb{E} [\nabla \log \pi \cdot (G_t - b(s_t))]$$

✓ 핵심 의미



편향 없음 (Unbiased):

Baseline은 상수처럼 작용하여 미분 시 0이 되므로, 원래의 목표인 Gradient 방향을 왜곡하지 않음.



분산 감소 (Variance Reduction):

보상의 절대 크기를 줄여($G_t - V(s)$) Gradient 추정치의 흔들림을 획기적으로 줄임.

Baseline
 $b(s)$ 상태 의존 기준값 (보통 가치 함수 $V(s)$)

Advantage Approx.
 $G_t - b$ 평균 대비 얼마나 더 좋은가?

Zero Mean
 $E = 0$ Score Function의 기댓값은 0임

Advantage Function 개념

행동이 평균보다 얼마나 더 좋은가? (상대적 가치 평가)

정의 (Definition)

$$A(s, a) = Q(s, a) - V(s)$$

특정 행동 a 를 했을 때의 가치 $Q(s, a)$ 에서 상태(s) 의 평균 가치 $V(s)$ 를 뺀 값.

핵심 의미



$Q(s, a)$

이 행동을 선택했을 때의 가치



$V(s)$

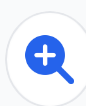
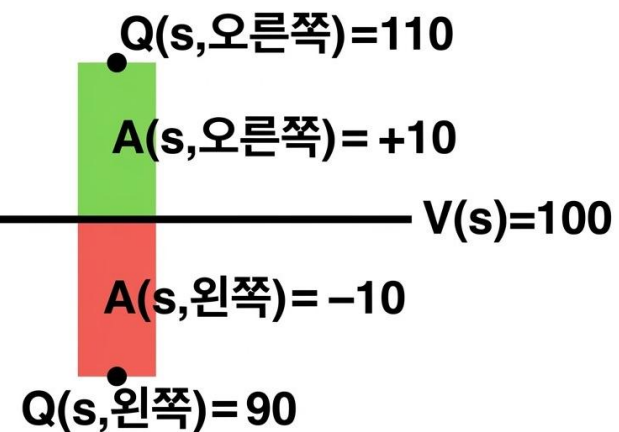
이 상태에서의 평균적인 가치 (Baseline)



$A(s, a)$

평균 대비 얼마나 더 이득인가?

가치(Value)



직관적 예시: $V(s) = 100$

평균 가치보다 높으면 양수(+), 낮으면 음수(-) Advantage를 가짐

Positive Advantage

+10

"Good Action"

Negative Advantage

-10

"Bad Action"

Advantage 계산방법(MC, TD, GAE)

Bias(편향)와 Variance(분산)의 트레이드오프를 고려한 선택



Monte Carlo

Unbiased (무편향) High Variance

$$A_t = G_t - V(s_t)$$

- 에피소드가 끝날 때까지 기다려야 함
- 실제 리턴값 사용으로 정확하지만 분산이 매우 큼
- REINFORCE 알고리즘에서 주로 사용



TD (1-step)

Biased (편향) Low Variance

$$A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

- 즉시 업데이트 가능 (Online)
- $V(s)$ 추정치에 의존하므로 초기 편향 존재
- 기본적인 Actor-Critic 구조



GAE (n-step)

Balanced Trade-off

$$A_t^{GAE} = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}$$

- λ 파라미터로 Bias-Variance 조절
- TD 에러 δ_t 의 가중 합계 사용
- PPO, A2C 등 현대 알고리즘의 표준

특징 구분	Monte Carlo (MC)	TD (1-step)	GAE(λ)
Bias (편향)	없음 (0)	높음	조절 가능 (λ)
Variance (분산)	매우 높음	낮음	조절 가능 (λ)
업데이트 시점	에피소드 종료 후	매 스텝 즉시	n-step 후 또는 종료 후
주요 사용	REINFORCE	Simple AC	PPO, TRPO, A2C

Actor-Critic 구조 (Actor-Critic Architecture)

정책(Actor)과 가치 함수(Critic)의 상호작용

Actor (배우)

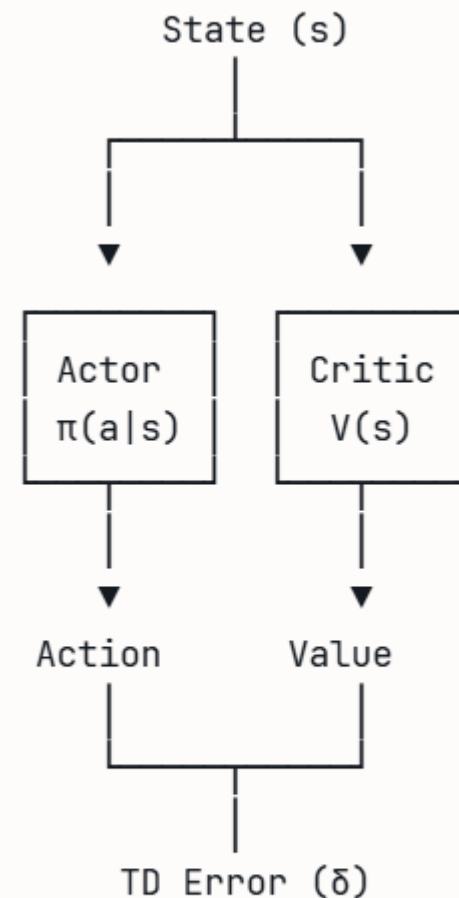
- ✓ 역할: 행동 선택 (Policy)
- 🎯 목표: Critic의 평가에 따라 확률 조정

Critic (비평가)

- ✓ 역할: 행동 평가 (Value)
- 💬 목표: TD Error 를 통해 Actor 피드백

Actor는 행동을 실행하고, Critic은 그 행동을 평가하여 피드백을 제공함

[아키텍처 다이어그램]



역할:

Actor: 행동 선택 (정책)

Critic: 행동 평가 (가치)

Actor-Critic 학습 과정

Online Update: 매 스텝마다 실시간 학습

1

▶ 행동 선택 및 실행

현재 상태에서 Actor 행동을 선택

2

⚙️ Critic 업데이트 (TD Error)

TD Target과 현재 가치 차이를 최소화

$$\text{TD Error: } \delta = r + \gamma V(s') - V(s)$$

3

⚙️ Actor 업데이트

TD Error 를 Advantage로 사용하여 정책 개선

δ를 Advantage로 사용

i 핵심 특징



Online Learning

에피소드가 끝나지 않아도 매 스텝마다 즉시 업데이트 가능



Low Variance

Critic이 가치를 추정하므로 REINFORCE보다 분산이 낮음



Bias Introduced

Critic의 추정값이 부정확할 경우 편향(Bias)이 발생할 수 있음

PYTHON PSEUDOCODE

```
# 1-step TD
td_error = reward + gamma * V_next - V_curr
# Update Critic
critic_loss = td_error.pow(2)
# Update Actor
actor_loss = -log_prob * td_error.detach()
```

A2C (Advantage Actor-Critic)

동기식(Synchronous) 업데이트와 정교한 Advantage 추정으로 학습 안정화



n-step Returns 활용

1-step TD 대신 n개의 스텝을 본 후 가치를 추정
Bias(편향)와 Variance(분산)의 균형을 맞추어 더 빠르고 안정적으로 수렴

$$G_t^{(n)} = r_t + \gamma r_{t+1} + \dots + \gamma^n V(s_{t+n})$$



병렬 환경 (Parallel Environments)

여러 개의 독립된 환경(Workers)에서 동시에 데이터를 수집합니다.
데이터 간의 시간적 상관관계를 끊어 학습 안정성을 크게 높입니다.



Entropy Regularization

목적 함수에 엔트로피 보너스항을 추가하여,
정책이 조기에 한 가지 행동으로 고착되는 것을 막고 탐색을 유도합니다.

⚡ A2C는 GPU 활용 효율이 높아 A3C(비동기)보다 더 선호됨

병렬 학습 구조



A2C vs Actor-Critic 비교

기본 구조에서 현대적 표준으로의 진화



Actor-Critic

기본적인 구조



1-step TD

한 스텝만 보고 바로 업데이트하여 편향(Bias)이 존재할 수 있음



단순한 구조

Advantage를 명시적으로 계산하지 않거나 단순 TD Error 사용



기본 성능

REINFORCE보다는 낫지만 여전히 학습이 불안정할 수 있음



동기식 업데이트

주로 단일 환경에서 순차적으로 학습 진행

성능 비교 계층도

REINFORCE



REINFORCE + Baseline



Actor-Critic



A2C



A2C (Advantage AC)

개선된 현대적 접근



n-step Returns

여러 스텝을 본 후 업데이트하여 안정성 향상 및 편향 감소



Entropy Bonus

목적 함수에 엔트로피 항을 추가하여 조기 수렴 방지 및 탐색 유도



더 빠른 학습

병렬 환경(Parallel Envs) 사용으로 데이터 수집 속도와 학습 효율 극대화



높은 안정성

Advantage Normalization 등으로 학습 과정을 더욱 안정화

Actor-Critic의 한계: 학습 불안정성

과도한 정책 업데이트로 인한 성능 붕괴 현상 (Policy Collapse)

정책 붕괴 시나리오

Episode 1000

잘 학습된 상태

Good Policy

Reward: 450

⚡ 너무 큰 Gradient Update 발생!

Episode 1002

정책이 망가짐

Collapsed

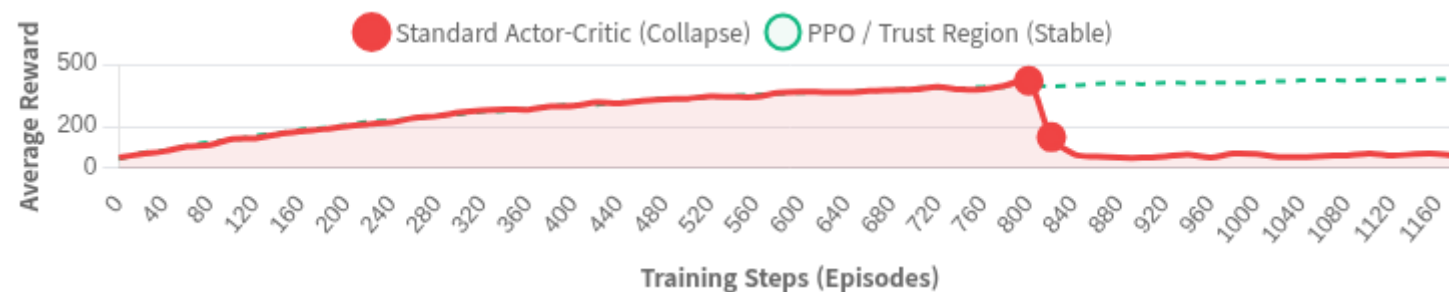
Reward: 50

원인 분석

한 번의 **과도한 정책 변화**가 이전까지 학습한 지식을 무효화 시킴.

나쁜 정책으로 수집된 데이터는 다시 나쁜 학습으로 이어지는 악순환 발생.

성능 급락 현상 (Catastrophic Forgetting)



Step Size Sensitivity

학습률(Learning Rate)에 매우 민감하여 튜닝이 어려움

Data Efficiency

한 번 망가지면 복구하는 데 막대한 데이터와 시간 소요

Solution Needed

정책 변화의 폭을 제한하는 Trust Region 기법 필요

Trust Region 개념

안전한 학습을 위한 정책 업데이트 제한 (조금씩만 변경하자)

핵심 아이디어

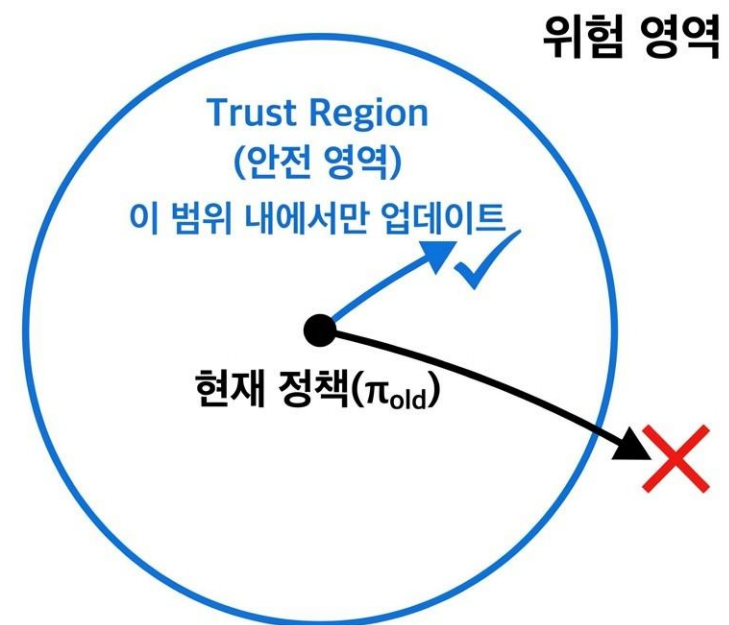
"정책을 너무 급격하게 바꾸지 말자"

급격한 정책 변화는 학습 붕괴(Collapse)를 초래할 수 있습니다.
현재 정책과 새로운 정책의 차이를 일정 범위 내로 제한하여
안정성을 보장합니다.

제약 조건 (Constraint)

$$KL(\pi_{old} || \pi_{new}) \leq \delta$$

- ✓ KL Divergence: 두 확률 분포의 차이 측정
- ✓ δ (Delta): 허용 가능한 최대 변화량 (예: 0.01)



PPO 핵심 아이디어: Clipping

복잡한 KL 제약 대신, 간단한 비율 제한(Clipping)으로 안전성을 확보한다



중요도 비율 (Importance Ratio)

이전 정책 대비 현재 정책의 확률 비율을 계산하여 정책이 얼마나 변했는지 측정합니다.

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$



Clipping 메커니즘

비율 r_t 가 $[1-\epsilon, 1+\epsilon]$ 범위를 벗어나면 강제로 잘라내어(Clip), 과도한 업데이트를 방지합니다. (보통 $\epsilon = 0.2$)



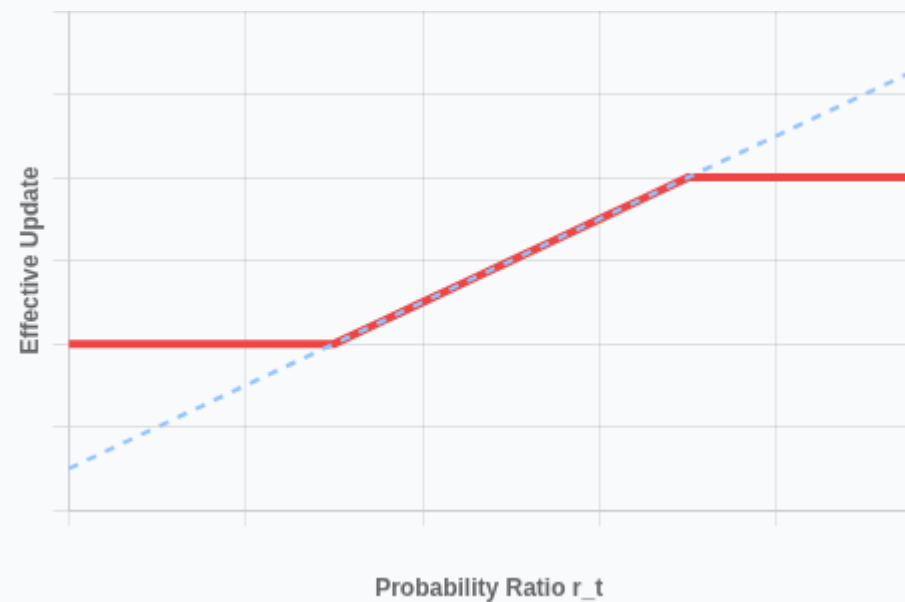
효과 및 장점

1차 미분만 사용하여 계산이 훨씬 빠르고 구현이 간단합니다.

PPO는 현재 강화학습 분야에서 가장 널리 쓰이는 표준 알고리즘입니다.

Clipping Function Visualization

비율(r_t)이 범위를 벗어나면 기울기가 0이 됨



● Unclipped Objective ● Clipped (PPO)



$\epsilon = 0.2$ 일 때, 0.8과 1.2 밖에서는 업데이트 제한

PPO-Clip 목적함수

안전하고 효율적인 정책 업데이트를 위한 Clipping 메커니즘

Clipped Surrogate Objective

The "PPO" Formula

핵심 수식

→

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

↑ Advantage > 0 (좋은 행동)

행동 확률을 높여야 하지만, **너무 많이 높이지 않음**
비율 r_t 가 $1+\epsilon$ 를 초과하면 Clipping → 보수적 증가

↓ Advantage < 0 (나쁜 행동)

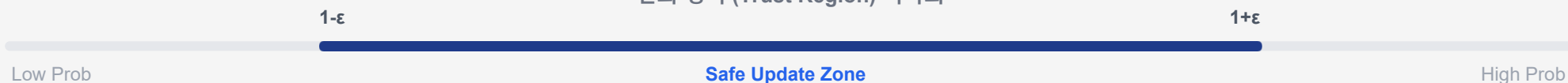
행동 확률을 낮춰야 하지만, **너무 많이 낮추지 않음**
비율 r_t 가 $1-\epsilon$ 미만이면 Clipping → 보수적 감소

r_t Probability Ratio
Importance ratio

ϵ Hyperparameter
Clipping 범위 (보통 0.1 ~ 0.2)

A_t Advantage
행동의 상대적 가치 (Q - V)

신뢰 영역 (Trust Region) 시각화



PPO 알고리즘 전체 과정

Proximal Policy Optimization: 안정성과 효율성의 조화

1. 경험 수집

- 여러 에피소드 실행
- 데이터 버퍼에 저장

2. Advantage 계산

- GAE 사용

3. 여러 Epoch 학습

- 같은 데이터로 여러 번
- 미니배치 사용

4. Clipping 적용

- 안전한 업데이트



PPO의 강점



Sample Efficiency

중요도 샘플링(Importance Sampling)으로 데이터를 여러 번 재사용하여 학습 속도 향상



Stability

Clipping을 통해 정책이 급격하게 변하는 것을 방지하여 학습 붕괴(Collapse) 예방



Simplicity

복잡한 2차 최적화 없이 1차 미분(First-order)만으로 구현 가능

PPO 실제 응용 사례

게임부터 로봇 제어까지: PPO가 보여준 압도적인 성능과 범용성



OpenAI Five (Dota 2)

PPO를 사용하여 세계 챔피언 팀을 상대로 승리.
대규모 분산 학습과 긴 시계열(Long Horizon) 문제를 해결

Massive Scale

Team Play



로봇 보행 및 조작

Boston Dynamics의 Atlas 등 4족/2족 보행 로봇의 동적 제어 학습에 표준 알고리즘으로 사용됩니다.

Continuous Control

Sim-to-Real

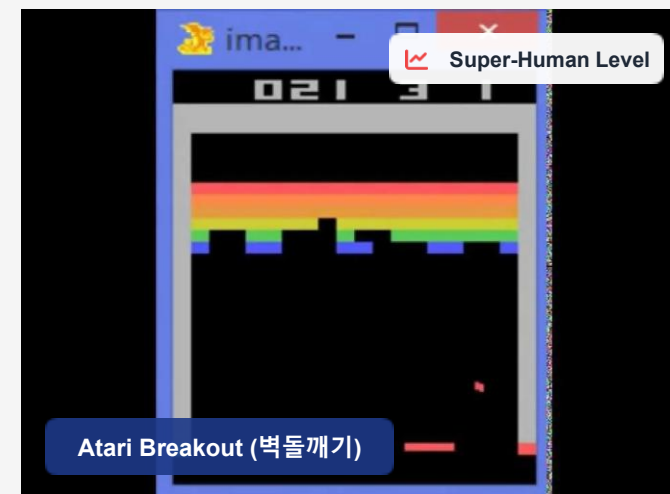
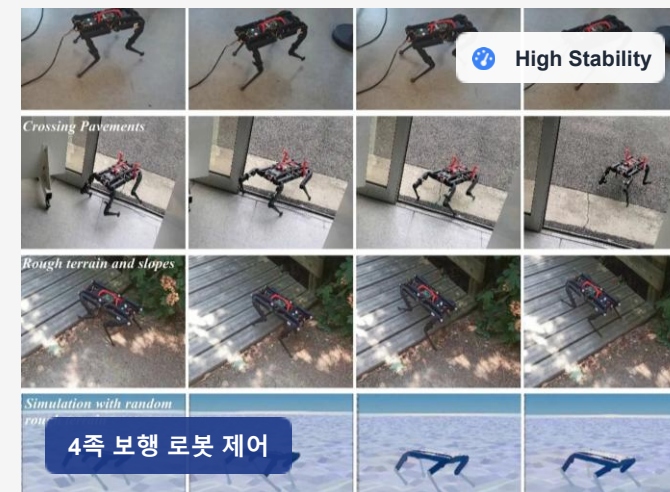
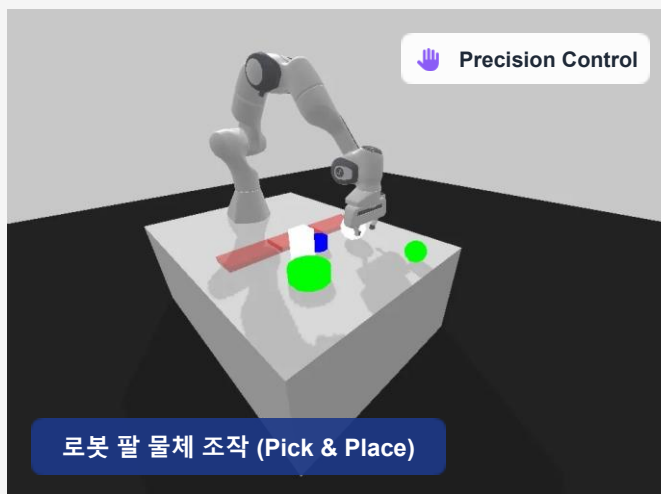
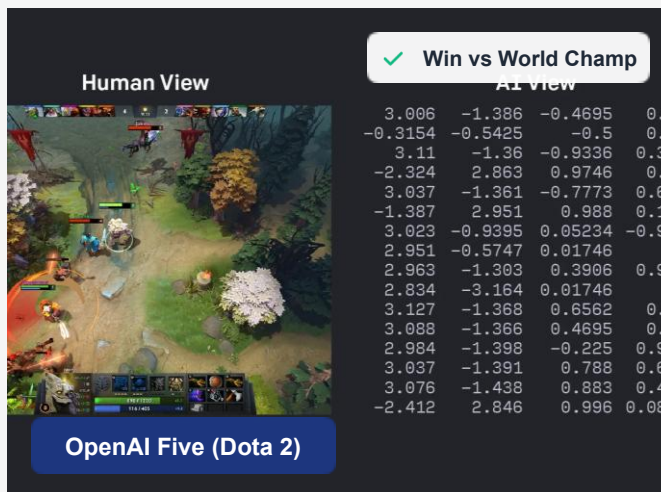


Atari & Arcade Games

DQN보다 더 안정적이고 빠른 학습 속도로 Atari 벤치마크에서 SOTA(State-of-the-Art) 성능을 기록했습니다.

Sample Efficiency

Stability



i PPO는 복잡한 전략 게임부터 정밀한 물리 제어까지 다양한 도메인에서 검증된 성능을 보여줍니다.

Policy Gradient부터 PPO까지: 현대 강화학습의 여정

Foundation

정책 기반 학습

- ✓ 정책을 직접 최적화
- ✓ 확률적(Stochastic) 행동 선택으로 탐색 용이
- ✓ 연속 행동 공간(Continuous Action) 처리에 적합

Stabilization

분산 감소 기법

- ✓ **Baseline:** 상태 가치 $V(s)$ 를 빼서 상대적 평가
- ✓ **Advantage:** $A(s,a) = Q(s,a) - V(s)$
- ✓ 평균보다 얼마나 좋은지(이득)만 학습 신호로 사용

Architecture

Actor-Critic 구조

- ✓ **Actor:** 행동 선택 (정책 네트워크)
- ✓ **Critic:** 행동 평가 (가치 네트워크)
- ✓ 온라인 학습(Online Learning)과 저분산 달성

Improvement

A2C (Advantage AC)

- ✓ n-step Return으로 편향과 분산 조절
- ✓ 엔트로피 정규화로 조기 수렴 방지 및 탐색 유지
- ✓ 병렬 환경 처리로 데이터 수집 및 학습 가속화

State of the Art

PPO (Proximal Policy Opt)

- ✓ **Clipping:** 정책 변화 비율을 일정 범위로 제한
- ✓ 학습 붕괴 방지 및 안정적인 단조 향상 보장
- ✓ 높은 샘플 효율성(Epoch 반복 학습)과 구현 용이성

Real World

실무 적용 표준

- ✓ **RLHF (ChatGPT):** 인간 피드백 기반 미세 조정
- ✓ **Robotics:** 보행, 조작 등 연속 제어 문제 해결
- ✓ 안정성, 효율성, 범용성에서 현재 최고의 선택

💡 결론: PPO는 복잡한 하이퍼파라미터 튜닝 없이도 준수한 성능을 보장하는 "Default" 알고리즘